🎯 Demonstration Outcome

✔️ Confidentiality through row-level filtering
✔️ Integrity through restricted modification
✔️ Accountability through audit logs
✔️ Context-awareness through session variables
✔️ Enterprise-style layered database security

If you want, I can now prepare:

- 🔥 Viva Q&A sheet (probable 15–20 questions)
- 📊 Architecture diagram explanation section
- 📑 Final report "Results & Discussion" section
- 🧠 Advanced features to impress evaluator

Tell me the next step.

just tell me in table format, what all roles have what querying permissions and who all can access what tables, and based on what

Perfect 👍
Here is the **complete permission summary** of your system in clear table format.

This shows:

- ✔️ Who can access which tables
- ✔️ What operations they can perform
- ✔️ Under what conditions
- ✔️ What enforces it (RBAC / RLS / Location)

You can directly put this in your report.

📃 1️⃣ PostgreSQL Privilege-Level Permissions (GRANTS)

Role	employees (SELECT)	employees (UPDATE)	employees (INSERT)	employees (DELETE)	audit_logs (SELECT)	Enforced By
Admin	✔	✔	✔	✔	❌	GRANT + RLS
HR	✔	✔	✔	❌	❌	GRANT + RLS

Role	employees (SELECT)	employees (UPDATE)	employees (INSERT)	employees (DELETE)	audit_logs (SELECT)	Enforced By
Manager	✔	✔	✔	❌	❌	GRANT + RLS
Employee	✔	❌	❌	❌	❌	GRANT + RLS
Auditor	❌	❌	❌	❌	✔	GRANT

Note:

- PostgreSQL GRANT only enables possibility.
- RLS decides *actual row access*.

📃 2️⃣ Row-Level Access (Virtual Private Database Behavior)

Table: `employees`

Role	Rows Visible	Condition
Admin	All rows	app.role = 'Admin'
HR	Only department rows	department = app.department
Manager	Only department rows	department = app.department
Employee	Only own record	emp_name = app.username
Auditor	None	No RLS policy

Enforced by:
✔ Row-Level Security (RLS) policies
✔ Session variable binding

📃 3️⃣ Location-Based Modification Control

Applies to: UPDATE / INSERT operations

Role	Internal Location	External Location	Controlled By
Admin	✔ Allowed	✔ Allowed	RLS
HR	✔ Allowed	❌ Blocked	RLS + app.location
Manager	✔ Allowed	❌ Blocked	RLS + app.location
Employee	❌ Not allowed anyway	❌	GRANT + RLS
Auditor	❌	❌	No write privilege

Condition used inside policy:

```
current_setting('app.location') = 'Internal'
```

📄 4️⃣ Audit Logging Coverage

Table: `audit_logs`

Event Type	Logged?	Triggered By
INSERT (employees)	✔️	AFTER INSERT trigger
UPDATE (employees)	✔️	AFTER UPDATE trigger
DELETE (employees)	✔️	AFTER DELETE trigger
SELECT	❌	PostgreSQL limitation
RLS Denied Attempts	❌	Blocked before trigger

📄 5️⃣ Context Variables Affecting Access

Context Variable	Purpose	Used In
app.role	Role evaluation	RLS policies
app.department	Department filtering	RLS
app.username	Self-access logic	RLS
app.location	Location-based restriction	RLS
session (login)	Identity binding	RBAC

📄 6️⃣ Final Layered Enforcement Model

Layer	Mechanism	What It Controls
Authentication	DB Role Login	Who connects
Authorization	GRANT statements	What operations are possible
Data Isolation	RLS policies	Which rows are visible
Context Control	Session variables	Dynamic access decision
Location Restriction	app.location	Modify permission
Accountability	Triggers + audit_logs	Who modified data

🎯 One-Line Summary Per Role

Role	What They Can Do
Admin	Full data access and modification (location-independent)
HR	Access department data, modify only when Internal

Role	What They Can Do
Manager	Access department data, modify only when Internal
Employee	Read own data only
Auditor	Read-only access to audit logs

This is your complete authorization + context + auditing matrix.

If you want, I can now:

- Convert this into a LaTeX-ready table section
- Create a clean architecture diagram explanation
- Prepare a viva rapid-fire Q&A sheet
- Or help you add time-based restriction

Tell me which one.

Create a clean architecture diagram explanation

Perfect 👌
Now I'll give you a **clean, professional architecture explanation** that you can:

- Put in your report
- Explain in viva
- Draw on board in 60 seconds

I'll give you:
1️⃣ Diagram structure (ASCII for report)
2️⃣ Component explanation
3️⃣ Security flow explanation
4️⃣ Short viva explanation version

🏗️ 1️⃣ High-Level Architecture Diagram

```
+---------------------+
|      User           |
| (Admin/HR/Mgr/etc)  |
+----------+----------+
           |
           | Login (DB Role)
           v
+---------------------------+
| PostgreSQL Authentication |
| (Role-based login)        |
+------------+--------------+
             |
             | Session Created
             v
+-------------------------------------+
| Session Context Binding Layer       |
|  SET app.role                       |
|  SET app.department                 |
|  SET app.username                   |
|  SET app.location                   |
+------------------+-----------------+
                   |
                   v
+-------------------------------------+
| Row-Level Security Engine (RLS)     |
|  - Role check                       |
|  - Department filter                |
|  - Self-access check                |
|  - Location check                   |
+------------------+-----------------+
                   |
           +-------+--------+
           |                |
           v                v
+----------------+   +-------------------+
| employees      |   | audit_logs        |
| (Protected DB) |   | (Forensic logs)   |
+----------------+   +-------------------+
           |
           v
+----------------------------+
| Audit Trigger Layer        |
| AFTER INSERT/UPDATE/DELETE |
+----------------------------+
```

🧠 2️⃣ Component Explanation

🔹 1. User Layer

Represents:

- Admin
- HR
- Manager
- Employee
- Auditor

They connect using PostgreSQL login roles.

Security Type:
✔️ Authentication layer

🔹 2. PostgreSQL Authentication Layer

Mechanism:

- CREATE ROLE
- Password-based login
- No application dependency

Purpose:

> Identify who is connecting.

Security Type:
✔️ Identity verification

🔹 3. Session Context Layer

Session variables:

```
SET app.role
SET app.department
SET app.username
SET app.location
```

Purpose:

> Provide dynamic contextual attributes for policy evaluation.

Security Type:
✔️ Context-aware attribute binding

🔹 4. Row-Level Security Engine (Core)

This is your VPD equivalent.

Mechanism:

- PostgreSQL Row-Level Security
- USING clauses evaluate:
  - app.role
  - app.department
  - app.username
  - app.location

Purpose:

> Filter rows before query execution.

Security Type:
✔️ Authorization
✔️ Data isolation

Important Concept:

RLS acts BEFORE query result is returned.

🔹 5. Employees Table (Protected Data)

Contains:

- Sensitive data
- Salary information
- Department information

Protected by:

- RLS policies
- GRANT constraints

🔹 6. Audit Trigger Layer

Mechanism:

AFTER INSERT / UPDATE / DELETE triggers.

Purpose:

> Log all successful modifications.

Security Type:
✔️ Accountability
✔️ Forensics

🔹 7. Audit Logs Table

Stores:

- user_id
- action
- table
- result
- timestamp

Accessible only by Auditor role.

Security Model:

✔️ Separation of duties

🔐 3️⃣ How Request Flows (Step-by-Step)

When a Manager runs:

```
UPDATE employees SET salary = salary + 500;
```

Flow:

1. PostgreSQL verifies login role
2. Session variables provide context
3. RLS evaluates:
    - Is role Manager?
    - Does department match?
    - Is location Internal?
4. If policy passes → Update allowed
5. Trigger fires
6. Audit record inserted
7. Result returned

If policy fails:

Step 4 blocks execution
Trigger does NOT fire
No modification happens

🛡️ 4️⃣ Security Layers Summary Diagram

```
Layer 1: Authentication        → DB Roles
Layer 2: Authorization         → GRANT statements
Layer 3: Context Awareness     → Session variables
Layer 4: Data Isolation        → Row-Level Security
Layer 5: Environmental Control → Location policies
Layer 6: Accountability        → Audit triggers
```

🎓 5️⃣ 30-Second Viva Explanation Version

Say this confidently:

> "The architecture separates authentication, authorization, and accountability. Users authenticate using PostgreSQL roles. Session variables bind contextual attributes such as role, department, and location. PostgreSQL Row-Level Security enforces data isolation dynamically at query execution time. All successful modification operations are logged via database triggers, ensuring traceability and separation of duties."

That sounds very professional.

📌 6️⃣ Why This Architecture Is Strong

✔️ Enforced entirely at database layer
✔️ Independent of application logic
✔️ Context-aware dynamic decisions
✔️ Supports principle of least privilege
✔️ Supports separation of duties
✔️ Provides forensic audit trail

If you want, next I can:

- Generate a clean draw.io style architecture diagram description
- Convert this into LaTeX figure with explanation
- Provide a security risk analysis section
- Or create advanced anomaly detection enhancement

Tell me the next move.